

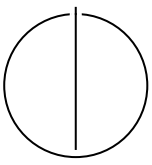
SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Generating Loop Termination Proofs for eBPF in LLVM

Deniz Mersinlioğlu





SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

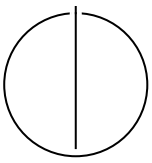
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Generating Loop Termination Proofs for eBPF in LLVM

Erzeugen von Schleifenterminationsbeweisen für eBPF in LLVM

Author:	Deniz Mersinlioğlu
Examiner:	Prof. Pramod Bhatotia
Supervisor:	Martin Fink
Submission Date:	21.09.2026



Acknowledgments

I would like to thank the following people for their part in the creation of this thesis:

My examiner Prof. Pramod Bhatotia for making this research project possible in the first place,

My supervisor Martin Fink for his guidance and proofreading,

My family for supporting me through my studies,

My friends Alex, Emilija, Gülşah and Tunç Batu for their help throughout my time in Germany,

And the professors and tutors of the Technical University of Munich whose lessons gave me the skills I needed to complete this thesis.

Abstract

Extended Berkeley Packet Filters, or eBPF for short, are a feature of the Linux kernel that allow users to add their own packet filters to the kernel, to be run in kernel mode, avoiding the computational cost of mode switches. However, in order to ensure that kernel-mode privileges are not abused, the kernel must verify each eBPF filter before it can be activated. However, the current approach used inside the kernel remains limited. For example, in order to verify that a for-loop terminates, the current verifier has to loop over every iteration of the loop manually, causing verification complexity to increase dramatically. To combat this, a formal eBPF verification infrastructure that separates the burdens of proof generation and proof verification is required, utilizing proof-carrying code to convey additional information to the verifier. In this paper, we showcase, implement and evaluate a series of augmentations to the LLVM compiler stack that carry metadata down the compilation chain, ultimately attaching loop termination proofs to the binary itself. We test our approach by checking if various types of loops can be proven to terminate in this new verification paradigm. The kernel-side implementation of the formal proof verifier is considered to be out of scope for this paper, and is the subject of another, concurrent research project at our Chair.

Zusammenfassung

Extended Berkeley Packet Filters, oder gekürzt eBPF, ist eine Funktion des Linux-Kernels, die Nutzern es erlaubt, eigene Paketfilter dem Kernel hinzuzufügen, wobei sie im Kernel-Modus laufen und dadurch den Rechenaufwand des Moduswechsels vermeiden. Jedoch muss der Kernel jeden eBPF-Filter verifizieren, bevor er aktiviert werden kann, um zu vermeiden, dass Kernel-Modus-Privilegien missbraucht werden. Der Ansatz, der in dem Kernel gerade dazu angewendet wird, bleibt aber begrenzt. Zum Beispiel muss das jetzige Verifikationsprotokoll jede Iteration einer Schleife manuell überprüfen, um zu verifizieren, dass eine For-Schleife terminiert, wodurch die Verifikationskomplexität dramatisch wächst. Um dieses entgegenzuwirken, eine formale Verifikationsinfrastruktur für eBPF, die die Last des Beweisgenerations und des Beweisverifikations voneinander trennt, ist nötig, indem Proof Carrying Code angesetzt wird, um zusätzliche Informationen an dem Verifizierern weiterzuleiten. In dieser Arbeit implementieren, evaluieren und präsentieren wir eine Menge von Erweiterungen für den LLVM-Compiler-Stack, die Metadaten durch das Kompilationsprozess weiterleiten und am Ende Schleifenterminationsbeweise an die Binärdatei selbst verknüpfen. Wir prüfen unseren Ansatz, indem wir überprüfen, ob die Termination verschiedener Arten von Schleifen mithilfe dieses neuen Verifikationsparadigmas bewiesen werden kann. Die Implementierung des formalen Verifizierers auf Kernel-Ebene liegt außerhalb des Rahmens dieser Arbeit und ist das Thema eines anderen, gleichzeitigen Forschungsprojekts an unserem Lehrstuhl.

Contents

Acknowledgments **iii**

Abstract **iv**

Zusammenfassung **v**

1 Introduction **1**
1.1 EBPF 1
1.2 The current verifier and its limitations 1
1.3 Our approach: SMT solver based verification 2
1.3.1 Using LLVM Analysis Information as Hints 2

2 Background **3**
2.1 eBPF 3
2.2 LLVM 4
2.2.1 LLVM Passes 4

3 Overview **5**
3.1 Motivation 5
3.2 Overview 5
3.2.1 Structure And Usage Of Our Proof Generator 5

4 Design **8**
4.1 Proving Loop Termination 8
4.2 Loop Measures 9
4.2.1 Loop Measure Grammar 11
4.2.2 Validity 12
4.3 Loop Measure Candidate Generation 13
4.4 Function tagging and Type Annotation 15
4.5 Multi-level information propagation 15
4.6 Fundamental Limitations of the Proof Generator 16

5 Implementation **17**
5.1 Pass-based Instrumentation . . . 17
5.1.1 Brief Overview of Implemented Passes . . . 17
5.1.2 Computing the measure . . . 18
5.1.3 Loop Measure Encoding . . . 19
5.2 PCSections . . . 21
5.2.1 PCSections as Inter-Pass Data Carriers . . . 21
5.2.2 PCSections as Output Medium . . . 23
5.2.3 Additional Scaffolding . . . 23
5.3 Testing Infrastructure . . . 24
5.3.1 Automated Parsing . . . 24
5.3.2 Unit Tests . . . 24
5.4 Practical Limitations . . . 24
5.5 Output Encoding . . . 25
5.5.1 Loop Tagging . . . 25
5.5.2 Function Type Tagging . . . 29
5.5.3 Instruction Tagging . . . 32
5.5.4 Register Tagging . . . 32

6 Evaluation **34**
6.1 Testing . . . 34
6.1.1 Criteria . . . 34
6.1.2 Simple Unit Tests . . . 34
6.1.3 Complex Unit Tests . . . 35
6.1.4 Edge-Case Testing Unit Tests . . . 38
6.1.5 Metadata Footprint of Test Cases . . . 40
6.1.6 Testing on production eBPF modules . . . 44
6.1.7 Results . . . 44
6.1.8 Notes on Loop Termination In Practice . . . 44

7 Related Work **45**
7.1 Loop Measures and Verification . . . 45
7.2 eBPF Proof Generation . . . 45

8 Conclusion **46**
8.1 Summary Of Results . . . 46
8.1.1 Practical Considerations . . . 46
8.2 Future Work . . . 46
8.2.1 Testing . . . 47

Abbreviations **48**

List of Figures **49**

List of Tables **50**

Bibliography **51**

1 Introduction

1.1 EBPF

In modern network facing production systems, the ability to process vast amounts of network packets in a flexible, fast and secure manner is critical. To ensure maximum flexibility, user-specified arbitrary packet-filtering programs are the gold standard. However, this leads to a tradeoff: allowing untrusted user binaries into kernel space that might crash the kernel is not an option for a secure system, while constant context switches between userspace and kernelspace would make a Filesystem in Userspace (FUSE)-analogous approach too inefficient. EBPF is the answer to this dilemma. To summarize, eBPF allows users to submit arbitrary eBPF bytecode to the kernel to be used as a packet filter, with the kernel verifying its safety and termination before allowing it to run (see Section 2.1). Since eBPF bytecode is Just In Time (JIT) compiled [Theb] into native machine code by the kernel, it achieves performance comparable to kernel-module packet filters [Sch+18], while the verification suite alleviates the security liability commonly associated with such an approach.

1.2 The current verifier and its limitations

Currently, the eBPF verifier implemented in the Linux kernel does not rely on any information external to the bytecode to do its work. Instead, it steps over the program, instruction by instruction, propagating preconditions via the well-defined instruction policy of the eBPF virtual machine [Theb]. To handle loops, every possible iteration of the loop has to be verified individually, to ensure that all relevant assumptions hold inside the loop for every loop iteration. [eBPb] To ensure that the verifier eventually stops, a limit is set on the complexity of the verification, approximately measuring the work the verifier has put in to verify the program so far. Once this threshold is crossed, verification fails automatically.

This approach, however, runs into very severe problems when verifying more complex programs, especially those including loops. Since loop verification requires an amount of effort roughly proportionate to the loop's maximum possible runtime, this means that long running loops may prove expensive or even untenable to verify. This is

due to information about the high level structure of loops being lost in compile time, meaning that the verifier has to explore all possible loop states to verify the loop. These limitations have motivated the development of several other novel eBPF verifier prototypes [Ham+25] [Ger+19], one of which uses Proof-Carrying Code (PCC) [Nec97]: [Thed].

1.3 Our approach: SMT solver based verification

To tackle the aforementioned limitations of the current eBPF verifier, we take on a formal verification based approach based on the PCC framework, where an SMT solver generates proofs during or after compilation, which can then be efficiently checked by the kernel during verification. The main implementation of this new verifier is the subject of a parallel research project at our Chair and is currently unfinished. In this paper, we take on a narrower subset of this problem: utilizing compile-time information from the LLVM compiler to analyze all loops in a given program, and generate proofs that they terminate.

1.3.1 Using LLVM Analysis Information as Hints

In order for a verifier to be able to efficiently prove termination and safety for a loop, it has to be given hints that point out important properties of the loop, such as the location of the loop variable, how the loop variable changes over time, as well as the boundary condition that determines when the loop halts. While automatic detection of such high level constructs in machine code remains challenging, these quantities are often readily available in LLVM's higher level compilation passes. This is what motivates the subject of this thesis: a proof generator embedded into the LLVM codebase itself, for maximum usability of LLVM's own loop analysis frameworks.

2 Background

2.1 eBPF

eBPF allows users to submit their own monitoring and packet filtering programs from userspace to be run in the kernel, after verification. These programs take the form of ELF binaries, containing eBPF bytecode, which is then verified and executed in the kernel. eBPF, being an interpreted but also JIT compilable architecture, runs in a VM-like environment similar to Java. This architecture has 10 registers, a fixed-size stack, and a RISC-like and restricted instruction set [Theb].

eBPF modules begin as source code, typically written in C (via libbpf), but potentially also Rust or Go. This source code is then compiled into eBPF's own assembly format, complete with its own Instruction Set Architecture (ISA), registers and instructions. This finished binary is distributed to user systems. When a privileged user (or an unprivileged one if unprivileged eBPF is enabled) submits this eBPF binary to the kernel, the kernel's own verifier checks to binary to ensure that the program can be proven to:

- Not contain dead code
- Not make memory accesses to memory regions it does not have access to
- Eventually terminate
- Not crash
- Not trigger various kinds of undefined behavior
- Uphold various other security guarantees that are out of scope for this paper

In order to prove these properties, the verifier steps over each instruction of the program, manually checking every possible execution path and loop iteration. Here, Abstract Interpretation is used to keep track of register values and how they evolve [Sha+25].

Once the program has been verified not to be hazardous to the kernel's security or functionality, it is accepted by the kernel and may begin running in kernelspace. There, it may hook into various syscalls, kernel tracepoints, and various other parts of the kernel to intercept data,

2.2 LLVM

LLVM is a compiler suite encompassing language frontends and hardware backends. Compilers implemented using LLVM implement frontends that tokenize, parse, and convert code to LLVM Intermediate Representation (IR), an intermediate representation language consisting of basic instructions. Although it is similar in structure to various kinds of assembly language, LLVM IR has no registers and is instead in SSA form. What sets LLVM aside from other compiler projects is this intermediate layer - compilers for multiple languages all end up emitting LLVM IR, which already has plenty of optimizations implemented for it, meaning that any compiler built upon LLVM may benefit from these optimizations. For more information, see [LA04].

However, the part that is the most relevant to us is the backend: for each ISA, the LLVM project implements a different, independent backend which takes optimized LLVM IR and lowers it into that ISA's machine code. This decoupling of the backend and frontend means that:

- When a new compiler is implemented using LLVM, it automatically gains support for all applicable backends implemented for LLVM, and
- Developers designing a new hardware backend only need to implement their backend once to implement support for all languages that have LLVM frontends.

2.2.1 LLVM Passes

The optimizations performed in the middle-end are implemented using so-called "passes" that may work on entire functions, basic blocks, loops or individual IR instructions. In the middle-end, these passes receive LLVM IR as input, perform analyses on it, and potentially modify it. The backend is also implemented in the form of passes, but here, we distinguish between "early" LLVM IR passes, which may still receive and utilize analysis information from higher-level LLVM passes (and critically, analysis data on loops), and "late" Machine IR passes, which happen at various stages of IR-to-machine-code lowering. Since machine IR passes are lower level, they have more information about the resulting machine code, such as register allocation information, but are also cut off from much of the IR-level analysis information.

3 Overview

3.1 Motivation

Currently, the eBPF verifier remains severely constrained in its ability to verify the guaranteed termination and safety of loops, since it has to check every possible iteration of every loop it encounters, and has a limited instruction count limit to ensure that the verifier terminates. These two factors, when put together, mean that loops with too many iterations may erroneously fail to verify, as the verifier effectively imposes the cost of unwrapping the worst-case loop execution upon itself and quickly runs out of capacity. To alleviate this, a more general way to reason about loops is required. To this end, we develop a set of augmentations for the LLVM compiler that leverage compiler insights to expose high-level loop structures to verifiers. The work discussed here focuses more narrowly on loop termination alone, and is meant to be a jumping-off point for future work that verifies eBPF programs more generally.

3.2 Overview

The main body of this paper is split into three sections: Design, Implementation and Evaluation. In the Design section (Section 4), we explain how loop measures may be used to verify loop termination in more general terms, and explain how our proof generator supplies loop measures. The Implementation section (Section 5) expands on how LLVM specific infrastructure and constructs are utilized by our proof generator to gather, propagate, and emit information through the various stages of compilation and into the resulting binary. Finally, in the Evaluation section (Section 6), we test our proof generator on various kinds of loops to see if it can generate loop measures that would theoretically be provably valid to a proof verifier.

3.2.1 Structure And Usage Of Our Proof Generator

The proof generator, being a compile-time utility with little reasoning ability of its own, is meant to be used in conjunction with a proof verifier during eBPF module development. For more information, see section 4.6. The following is how we envision the development workflow to be structured:

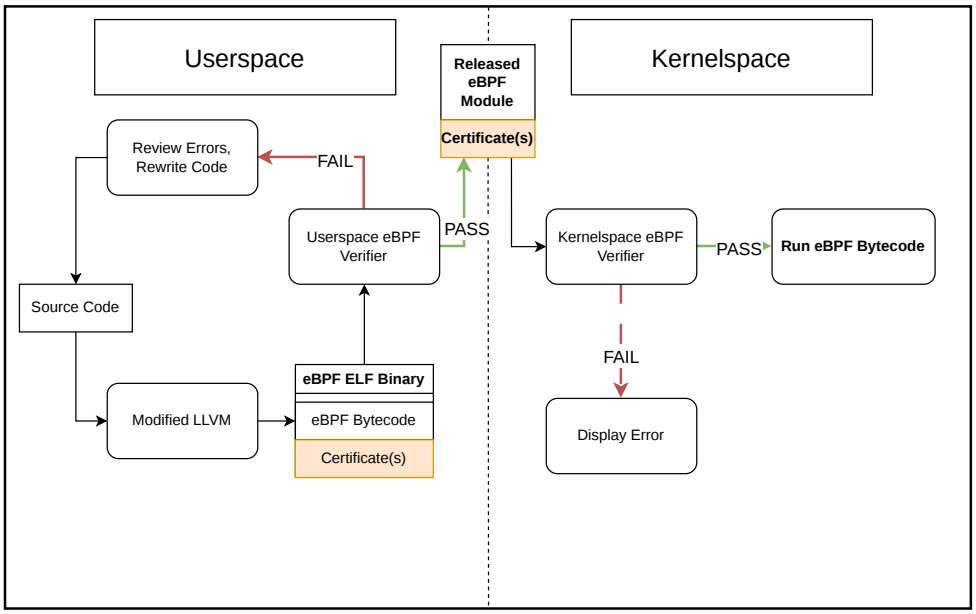


Figure 3.1: Overview of our envisioned workflow

Here, we assume that the eBPF Verifier which verifies our termination proofs will have a trusted core which is runnable in both userspace and kernelspace, producing identical results in both environments. The developer then uses our augmented compiler to compile their code, resulting in a proof-carrying binary. Their build system (Makefile, CMake, etc.) automatically submits the resulting binary to a local userspace version of the verifier, to check if the given proof is valid and proves the program's safety and termination. If verification fails, the developer takes the generated errors into account and rewrites their program, repeating the process until the program successfully verifies. Once verification succeeds, the resulting binary may be published and distributed. The end-user may then submit the released binary to their kernel for verification. Since the userspace verifier used by the developer and the kernelspace verifier on the end-user's end are both identical, the released binary is all but guaranteed to pass the kernelspace verifier and be prepared for execution. On the event that the binary fails to verify, an appropriate error message may be emitted to prompt the user to report this to the eBPF module's developers.

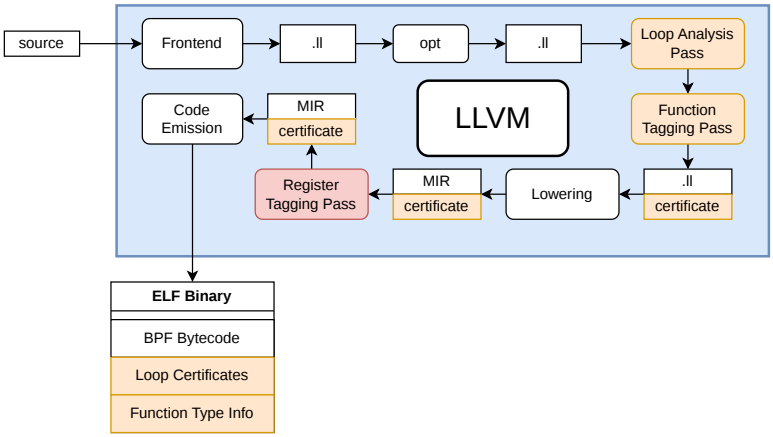


Figure 3.2: Overview of our implementation

The proof generator itself is structured as follows:

Initially, the eBPF program's source code is transpiled via its language's frontend to LLVM IR and IR optimizations are performed as usual. Once the frontend and middle-end optimizations are complete, the high level passes annotate the loops and functions with loop certificates and type signatures, respectively. Then, these annotations are persisted through lowering to machine IR and register assignment. Once register assignment is complete, a lower level pass adds register allocation information to the loop certificates. The resulting code and certificate bundle is then emitted into the ELF binary, with the bytecode being identical to what would have been generated without our modifications. The annotations our proof generator adds are bundled in the ELF file as discrete, separate ELF sections. These are what the verifier is expected to parse, in conjunction with the bytecode.

4 Design

In this section, we broadly detail a set of compiler modifications for extracting and embedding metadata about loops that can be used by a verifier to verify that a loop terminates. To this end, we specify a method to generate and embed loop certificates in the form of so-called "loop measures" (See section 4.2) into a program's metadata, with each certificate providing enough information to verify that a given loop terminates. We briefly touch upon how these certificates can be generated using a given modern compiler's loop analysis infrastructure in section 4.3, and how compilation information from multiple stages of the compilation pipeline may be utilized to generate various kinds of metadata (See section 4.5).

4.1 Proving Loop Termination

In order to prove that a given eBPF program terminates, we must prove that every backedge in the program is taken a finite amount of times. In order to prove this, our approach is to assign each backedge to a loop, and then prove that every loop in the program runs a finite number of iterations. Since a verified eBPF program also may not clobber its own return address and function call depth is limited to 8 [eBPa], we may assume that proving every loop in the program terminates is sufficient to prove that the program terminates.

To prove that a loop terminates, we must prove that the loop body is executed a finite number of times. For example, the loop

```
for (unsigned int i=0;i<100;i++) {
    ...
}
```

is constrained to 100 executions of its loop body, since i takes on every value in $[0, 99]$ exactly once. On the other hand, the loop

```
for (unsigned int i=0;i != 1;i += 2) {
    ...
}
```

will never terminate, because an increment of 2 will never set i's least significant bit, meaning that i takes on every even value available to an *unsigned int* infinite times.

In order to handle as wide a variety of loops as possible, we base our approach on quantifying the remaining number of loop iterations at any given point. For example, in the first loop, we may express the number of remaining loop iterations as $100 - i$, and notice that it is finite, proving mathematically that the loop will execute a finite number of times.

A more general version of this approach is what motivates our definition of Loop Measures, as follows:

4.2 Loop Measures

To quantify loop execution limits, we draw inspiration from a concept known as the W-Function, introduced in [Flo93]. Briefly, the aforementioned paper proposes assigning each edge in the program's execution graph with a so-called "W-Function", mapping the program state at the time of the edge's program execution to a value in a well-ordered set. The paper then imposes the restriction that each subsequent visit of that edge must result in a W-function value that is lower according to the well-ordered set's ordering. The assertion is that since a set being well-ordered is incompatible with the existence of an infinite sequence of values that is strictly descending based on the set's ordering, the existence of a W-function on that edge proves that the edge will be executed a finite number of times.

For the purposes of this paper, we build on this definition to define the "loop measure", a computable function assigned to the program edge(s) directly before the start of the loop, right after the loop condition is checked. Like the W-Function, our measure is placed at this point to guarantee that the program edge it is assigned to is executed a finite number of times, meaning that the loop itself is executed a finite number of times. Unlike the W-Function, however, we only focus on the entry points of loops, instead of labeling every edge in the program, and we constrain our definition to the well-ordered set $\mathbb{N}_0$ and the natural ordering for simplicity. This also allows for an elegant interpretation of our loop measure: its value is the maximum number of loop iterations that may be performed during the current execution of the loop, given a program state.

A loop measure ρ must uphold the following properties:

$$\text{Let } I \in \mathbb{N} \cup \{+\infty\};\ I \stackrel{\text{def}}{=} \text{Total loop iteration count} \tag{4.1}$$

For ease of reading, we set:

$$\rho_i = \rho(State_i) \tag{4.2}$$

with $State_i$ being a shorthand for the accessible program state on the ith execution of the loop header. One execution of the loop, from the start of the header to the end of the loop body (including the loop body itself), followed by a jump back to the loop header, transforms $State_i$ to $State_{i+1}$.

The loop measure must be a finite integer:

$$\rho_i \in \mathbb{N}_0 \; \forall i \in [1; I] \tag{4.3}$$

In the loop body, the measure must be positive.

$$\rho_i \geq 0 \; \forall i \in [1; I] \tag{4.4}$$

On each back-edge, i.e., when we execute the loop body one more time, the loop measure must strictly decrease.

$$\rho_i > \rho_{i+1} \; \forall i \in [1; I-1] \tag{4.5}$$

A loop measure ρ that satisfies all of these properties is henceforth referred to as a "valid" loop measure. A function that fails to fulfill one or more of these properties is, therefore, an "invalid" loop measure. A loop that possesses a valid loop measure can be assumed to terminate.

For example, the loop

```
for (int x=100; x>0;x--) {
    ...
}
```

could have its termination proven via the measure $\rho(State_i) = State_i(x)$, or $\rho = x$ for brevity.

However, the loop

```
for (unsigned int x=0; x<=UINTMAX;x++) {
    ...
}
```

will have x overflow and take on every single possible value an infinite number of times. More precisely,

$$State_i(x) = State_{i+2^{32}}(x) \tag{4.6}$$

meaning that since any valid measure must depend solely on x due to the loop condition operating exclusively on x:

$$\rho_{i+2^{32}} = \rho(State_{i+2^{32}}(x)) = \rho(State_i(x)) = \rho_i \tag{4.7}$$

which contradicts ρ_i being strictly monotonically descending. Hence, no valid loop measure can be generated.

4.2.1 Loop Measure Grammar

In order to specify a loop measure and certify its validity, we generate and embed loop certificates in the following format:

$$\langle program_proof \rangle ::= \overline{function_pointer \mapsto \overline{\langle loopcert \rangle}} \tag{4.8}$$

$$\langle loopcert \rangle ::= \{\textbf{iv_register} = reg(r); \textbf{loopid} = id; \textbf{measure} = \langle measure \rangle; \textbf{latches} = \overline{\langle latch \rangle}\} \tag{4.9}$$

$$\langle latch \rangle ::= pc \tag{4.10}$$

$$\langle measure \rangle ::= iv - \langle end \rangle \mid \langle end \rangle - iv \tag{4.11}$$

$$\langle end \rangle ::= \langle expr \rangle \tag{4.12}$$

$$\langle expr \rangle ::= (\langle operand \rangle; \overline{\langle expr \rangle}) \mid (udiv; \langle expr \rangle, \langle expr \rangle) \mid (\langle typedoperand \rangle; \langle expr \rangle, \langle type \rangle) \mid instr(pc) \mid function_arg(num) \mid const(val, \langle type \rangle) \mid vscale \tag{4.13}$$

$$\langle operand \rangle ::= + \mid - \mid * \mid umax \mid smax \mid umin \mid smin \mid phi \tag{4.14}$$

$$\langle typedoperand \rangle ::= ptrtoaddr \mid ptrtoint \mid trunc \mid sext \mid zext \tag{4.15}$$

$$\langle type \rangle ::= int(size) \tag{4.16}$$

Here, the signifier $\overline{item}$ means that there may be zero, one or multiple *item* entries present here. The operator $a \mapsto b$ means a mapping of a to b.

Proof Format

A program proof is a mapping from function pointers to loop certificates. This means that each function carries the loop certificates for all loops in its body. These loop certificates each consist of one loop measure (*measure*), as well as the register number of the identified loop variable (*iv_register*), a list of all backedges that are part of this loop (*latches*) and a unique integer ID number (*loopid*). The backedges are expressed as file offsets to the relevant machine instructions.

The measure takes the form of a difference between the loop variable *iv* and a loop-invariant *end* value. This *end* value, also referred to as the boundary value, is generally the value that the *iv* reaches when the loop has terminated. However, it may take any invariant value that ensures the validity of the overall measure.

This boundary value takes the form of an arbitrarily complex arithmetic expression, operating on:

- Result values of instructions that have executed before the loop's beginning, expressed as $instr(pc)$, with pc being the file offset to the instruction.
- Arguments that the function was called with, expressed as $function_arg(num)$, with num being the argument number.
- Arbitrary integer constants, expressed as $const(val, \langle type \rangle)$.

This expression may consist of the following operators:

Operators

The operand *udiv* stands for unsigned division. *umax* stands for unsigned-maximum, *smax* for signed maximum, and *umin* and *smin* are defined analogously. The *phi* operand is analogous to an LLVM PHI node, but with the entry block associations stripped out. It means that the result here may take on any of its arguments' values. The expression *vscale* is a hardware-intrinsic value that is unknown at compile time but constant during runtime, relating to scalable vectors. The typed operands *ptrtoaddr* and *ptrtoint* work as typecasts that cast the given pointer value to a type specified in the operands. *trunc* truncates a given integer value to a smaller integer type. *sext* and *zext* represent signed-extension and zero-extension operations respectively.

4.2.2 Validity

In order for this structure to be considered a valid proof, the following properties must additionally be satisfied:

- **loopid**s must be unique within each ELF module
- Each $\langle latch \rangle$ program counter must correspond to a branch or jump instruction pointing to the loop header, and all branches or jump instructions that may decrease the PC after execution must be tagged as latches in at least one loop. Here, the term "latch" is used interchangeably with the term "backedge", and refers to any instruction that may redirect execution flow back to the loop header.
- All registers referenced as $reg(r)$ must correspond to valid register numbers, meaning $1 \leq r \leq 24$, see Table 5.21. [1]
- The program counter referenced by each $instr(pc)$ must point to an instruction which has a well defined result value and destination register.

[1] Note: eBPF contains only 12 registers, from R0 to R11. However, it is also possible to access these registers' lower 32 bits, when accessing them via 32-bit instructions. In this case, the registers are labelled as W0-W11 and these usages are assigned separate register numbers by our proof generator.

- The value $\langle end \rangle$ must be a loop invariant.
- All included loop measures must consist of expressions whose values are available within the program state at the start of the loop.
- All included loop measures must be valid.

4.3 Loop Measure Candidate Generation

In order to generate these certificates, information from both high level loop analysis passes and low level register allocation passes is required. Our general approach is to implement our certificate generator in multiple compiler passes, with the higher level passes generating initial metadata structures containing the measure, and lower level passes supplementing the generated measure with information about registers. In the case of LLVM, our modifications span from the LLVM IR section of the eBPF backend to the very end of the backend's code emission phase. One important note is that we deliberately avoid making any edits to any frontend, confining all implementation to the backend. This ensures that all features we implement work in a language agnostic manner, generating valid loop certificates for any language that LLVM can compile into eBPF bytecode.

In the case of the higher end passes, we make heavy use of LLVM's loop analysis infrastructure and Scalar Evolution infrastructure to also be able determine measures in cases where the loop's endpoint is variable depending on what comes before the loop itself, and may be arbitrarily complex. For more information, see Section 5.1.2

In order to be able to derive a valid measure, we must have access to the loop's boundary condition. In LLVM, this boundary condition manifests as an end-value, to which the loop-variable is compared. The boundary value is calculated such that if $iv = end$, the loop body does not run again. For example, the loop header:

```
for (int i=0;i < 5000; i++)
```

leads to the end-value 5000. However, the loop header:

```
for (int i=0;i <= 5000; i++)
```

instead produces the end-value 5001. However, one must be conscious of LLVM's low-level optimizations. For example, the loop header:

```
for (int i=5000;i > 0; i--)
```

may be optimized by LLVM to run the decrement operation directly before the loop body instead of after, shifting the initial value of the loop to 5001 and the end-value to 1. In cases such as this, the end-value we produce corresponds to the comparisons

being done on the machine code level, and may not always match the original source code.

Due to the nature of its construction, it is possible for our measure to express an arbitrarily complex loop boundary-condition in terms of constant values and instruction dependencies. For example, the following code snippet:

```
int a = (Unknown);
int n = 10;
if (a) {
  bpf_printk("branch␣side␣effect\n");
  n = 20;
}

for (int i=0;i<n;i++) {
  bpf_printk("phi␣loop␣!=\n");
}
```

will cause the end-value to depend on the result of a PHI operation, taking on either 10 or 20 depending on which block the loop was entered from. In practice, this means that *end* is encoded as $(phi\ \%1\ \%2)$, with the corresponding dependencies set to $\%1 = 20$ and $\%2 = 10$ respectively. Note that we do not include which of these possibilities results from which incoming edge, as PHI nodes generally resolve to pure register-allocations and basic blocks may merge and lose their branch instructions. For PHI nodes specifically, this leaves the verifier with few options other than taking the most permissive possible option for the measure (in this case, $end = 20$, because the measure $10 - iv$ being valid implies that $20 - iv$ is also valid - not necessarily the other way around). However, in cases without PHI-adjacent effects, the result may be more useful. For example:

```
int a = (Unknown);
for (int i=0;i<a;i++) {
  bpf_printk("phi␣loop␣!=\n");
}
```

causes *end* to resolve to %1, with the corresponding dependency entry being a reference to the instruction in which the value of a is calculated. In this case, for instance, a verifier could potentially prove that $a < INTMAX$, meaning that the state $i = a$ is reachable, which in turn means that the loop will terminate. This assertion, however, requires more to prove or disprove than is available to us inside LLVM alone, see section 4.6.

One particular limitation for this approach is that loop measures can only be gen-

erated if LLVM's loop analysis tools find well-behaved loop bounds and an invariant stride. For example, the following loop:

```
int *unk = (Pointer to array of at least 20 ints);
for (int i=0;i<20;) {
    bpf_printk("side␣effect␣%d\n", i);
    if (unk[i] & 1) {
      i++;
    } else {
      i += 2;
    }
}
```

cannot be assigned a measure by our test suite, because the loop variable does not have one single linear stride for every iteration of the loop, resulting in LLVM being unable to give us any loop boundaries, including the end-value. However, this loop should theoretically still possess the measure $20 - i$, because i increments by at least 1 on each iteration, and the increment operations will not overflow due to $i < 20$ holding on each loop entry and therefore also during the increment steps.

4.4 Function tagging and Type Annotation

In addition to loop-level proofs, our annotation suite additionally emits type information for every function in the program, as additional aid for the verifier. Types are encoded recursively, with composite types such as structs, function signatures, and arrays containing type encodings for their constituent subfields. This is what allows function-argument dependencies of the measure strings to be well typed. See Table 5.10 for more information.

4.5 Multi-level information propagation

Since our metadata needs to have both high level information about loop structure and low level information about register allocation, the proof generation suite must be distributed across multiple levels of the compiler.

For example, in order for the part of the proof generation process that resides in the lower levels of the compiler to be able to meaningfully deduce the physical registers in which the loop variables reside, the higher-level instrumentation has to annotate the loop variable in a way that survives the lowering to machine instructions until the exact moment physical register allocation occurs, as can be seen in section 5.1.2. The main

difficulty with this is that LLVM IR itself is in SSA mode, while machine instructions use registers, meaning that the concept of a "value" as defined in LLVM IR does not have a direct counterpart in the resulting machine code. Furthermore, although our passes are all scheduled to run after all major middle-end optimizations are complete, translation to machine IR and then to machine code may still alter the structure of the program by deleting, merging or splitting certain instructions.

To make it possible for higher level passes to pass on information to lower level passes, we use an LLVM-specific tool called PCSections (see section 5.2) whose basic functionality is to allow annotation of either functions or instructions with arbitrary metadata consisting of constant data. These annotations are then preserved across translations, allowing information tagged onto LLVM IR instructions to persist into machine IR, so that lower level passes may intercept and augment it.

4.6 Fundamental Limitations of the Proof Generator

The main caveat of our approach is that, as seen in section 4.3, some loops may generate measures whose validity may depend on more general properties of the program. In this cases, checking the validity of the proof may require full SMT-solver capabilities. Lacking these capabilities, our proof generator currently generates false-positive proofs for loops that may not actually terminate. Since this means that the existence of the proof metadata may not necessarily imply loop validity, we skip most validity checks on the measure inside our proof generator, relying fully on the SMT solver's verification abilities.

To avoid both code-redundancy with the planner kernel-level proof verifier and sidestep the possibility of the proof-generator and kernel-level verifier falling our of sync, we recommend integrating this proof generator into a development pipeline as detailed in Figure 3.1.

5 Implementation

eBPF, like many other instruction sets, possesses an LLVM backend, which allows developers to write their eBPF modules in C and compile it to eBPF bytecode via clang. However, due to the aforementioned decoupling between the backend and the frontend, eBPF modules could theoretically be implemented in any language with an LLVM frontend, such as Rust or Go. This is what motivates our choice of LLVM for our proof generator: by keeping our proof generation logic constrained to the eBPF backend, we keep our proof generator language agnostic, while also making use of the mature pass management and loop analysis infrastructure of LLVM to gather information for our proofs.

In this section, we implement a set of passes in LLVM that generate and emit loop measures as discussed in Chapter 4. Our approach involves one high-level that gathers loop analysis information, one pass that annotates functions with type information, and one low level pass that adds register mapping information to our loop measures. [1]

5.1 Pass-based Instrumentation

In order to generate the metadata detailed in the design specification, our compiler augmentations mainly rely on LLVM passes at various stages of compilation to annotate instructions and functions at various stages of compilation.

For our implementation, we decided to keep all edits between the LLVM IR level passes and the eBPF backend's pre-emit phase. This means that our verifier is language agnostic.

5.1.1 Brief Overview of Implemented Passes

Our implementation is mainly divided into four passes. The bulk of the proof generation happens in the BPFLoopTagging pass, which iterates over high level LLVM IR loops and calculates their measures, attaching the resulting metadata to the resulting functions. The pass BPFFunctionTagging iterates over LLVM IR functions, tagging each one with type annotation information.

[1] Our proof generator is available at `https://github.com/TUM-DSE/llvm-tnbpf`

Once the high level passes are complete, LLVM IR is lowered to Machine IR. Directly after physical register allocation, the BPFIVRegisterMapping pass determines which physical registers each loop's loop variables are stored in, and appends this information to the loop certificates. As a final step, the BPFPCSectionFixup pass prunes any PCSections that may have been duplicated during the LLVM to Machine IR lowering process.

5.1.2 Computing the measure

As detailed in the design section, the two main quantities required to compute the measure are the loop variable and the end-value of the loop. To deduce the end boundary, we rely on LLVM's own loop analysis infrastructure as well as Scalar Evolution (SCEV).

Computing the Boundary

In order to compute a given loop's boundary value, we utilize SCEV to obtain an expression for a complex boundary-value that nevertheless stays invariant inside the loop itself. Since the boundary value for the end can depend on what happened before the loop, the SCEV value for said boundary can still have complex expressions like PHI nodes, without violating the loop invariance property we require. The result is an AST graph that we print recursively, using a lightly altered version of SCEV's own debug-printing code.

As mentioned in section 5.1.3, we make sure to print any instruction-dependencies, integer constants and function argument references as dependency references, including either references or the constants themselves in the dependency table to make the value legible to a proof checker.

Making the loop variable accessible to the proof checker

Once the loop's termination boundary has been calculated, all that remains is to ensure that the proof checker has a way of knowing where the loop variable is stored. To this end, we first use LLVM's loop analysis infrastructure to obtain a reference to the PHI node that collects the loop variable's current state at the very start of the loop header. This PHI node is then tagged with the loop's unique ID using a specialized, implementation-internal PCSection. In order to ensure that this tag metadata makes it to the PHI elimination phase of the compilation process, we patch the machine IR emitter to ensure that the temporary machine PHI instruction is emitted with the IR PHI node's PCSections. This allows the PHI elimination pass, which is the point in which a virtual register is first assigned to the result of the PHI node, to associate this

virtual register with the loop ID. At this point, the register-loop connection is made and no further references to other instructions are required. Therefore, in order to ensure that this information survives, the PHI elimination pass is augmented to tag the function itself with a PCSection matching the loop IDs to the virtual registers.

This table is then iterated over by a later lower-level pass with access to the register allocation table, which replaces the virtual register numbers with their corresponding physical register IDs. The result is a table mapping each loop's loop variable to a machine register.

Expressing the measure in terms of the induction variable

In order to make the printing process as clean as possible, we express the resulting measure in its entirety, including the loop variable itself, as one single SCEV expression that is recursively printed out. However, in doing so, we run into a problem: since SCEV normally seeks to generate expressions that can be computed from the loop iteration number, it automatically detects when a value being referenced is the loop variable and replaces it automatically with an expression depending on the loop iteration number. Since this substitution would require the generated structure to then include a way to compute the loop iteration number from the loop variable, which in many cases just generates a string that is massively similar in structure to the measure string itself, we simply wrap the loop variable in a special unknown-value class preventing SCEV introspection. Our printer then compares each value reference it comes across with the known loop variable reference, handling it as a special case and generating a loop variable indicator.

5.1.3 Loop Measure Encoding

In practice, this data format is embedded into the ELF binary in the form of multiple types of ELF sections, each with their own syntax. For each ELF module produced in the compilation process, a separate set of sections may be generated, in the case of the relevant section having at least one emitted entry. All generated section names have the following format:

$$\langle module_name \rangle_\langle section_name \rangle$$

With $\langle section_name \rangle$ being one of:

- "*loopdb_symbounds*", formatted according to Table 5.4
- "*loopdb_class*", formatted according to Table 5.1

- "$functiondb$", formatted according to Table 5.10
- "*iv_db*" , formatted according to Table 5.22
- "*sym_instr_db*", formatted according to Table 5.20

For each loop in the program, the main metadata structure that must exist for successful verification is the "*loopdb_class*" table, with entries for each function. If an entry for the loop is found inside this table, then the *hasTranslationTableMeasure* field may be checked - if true, then the *loop_id* and module name associated with this loop can be used to locate a matching metadata table in the relevant "*loopdb_symbounds*" section. This section contains the loop measure and all dependencies needed to compute it. The remaining tables are auxiliaries: "$functiondb$" stores type information on every function in the module, "*iv_db*" stores register register IDs for each loop's loop variable, and "*sym_instr_db*" stores location information for specific tagged instructions, whose output values or positions are referenced by other metadata tables.

In order for the main tables and auxiliary tables to refer to each other, two unique ID number types are used:

- For each loop that the instrumentation suite emits a certificate for, a module-unique loop id is assigned.
- In order to refer to expressions of the form $instr(pc, reg)$, the table *loopdb_symbounds*" uses instruction IDs, which are module-unique and refer to tagged instructions in the "*sym_instr_db*". Similarly, *loopdb_class* also uses instruction IDs to refer to loop latches, which are tagged in *sym_instr_db*.

Measure String

The measure is encoded as a string, built out of three main components:

- A reference to the loop variable, encoded as $\%iv$
- References to dependencies, encoded as $\%(dependency\ number)$
- Various mathematical operations upon these dependency entries, see section 4.8

Since the endpoint of the loop may depend on arbitrary constants, values computed during program execution, or on function arguments, these are encoded as dependencies in the header, referred to by their offset in the dependency table. This makes the resulting expression computable to a proof checker.

5.2 PCSections

The largest hurdles of the implementation are passing metadata from one pass to the next, and emitting it into parseable sections at the end. For both of these problems, we utilize the PCSections framework provided by LLVM. To briefly summarize, PCSections are a special kind of metadata that can be attached to either LLVM instructions (both LLVM IR and Machine IR) or functions that may contain arbitrary constant values, such as integers, anonymous structs and constant tuples. PCSection metadata consists of two parts: the desired section name and the data. The section name determines which ELF section the generated entry lands in. Once the instruction or function makes it to the binary-emitting phase of compilation, the following occurs:

- If the required ELF section does not exist yet, it is created.
- If the entity to be emitted is a function, an offset pointing to the function's beginning, followed the function's length, are appended to the end of the ELF section referred to by this PCSection.
- If the entity to be emitted is an instruction, an instruction offset is appended instead.
- Directly after this pointer, the constant data included in the PCSection is appended.

The size of each entry is platform dependent, but for eBPF, all entries mentioned in this list are 32 bits wide. Once assigned to an instruction, PCSection metadata may also be altered or removed by future passes.

5.2.1 PCSections as Inter-Pass Data Carriers

Since our proof needs to have metadata from both high level and low level passes, we first tag relevant instructions, as well as tagging functions with loop IDs and virtual registers in which the given loops' loop variables reside at the start of each loop body execution. This then allows the lower level passes to loop over the virtual registers and replace them with physical registers.

Modifying PCSections

Within LLVM's own metadata infrastructure, PCSections are stored in the following format, for a given PCSection S:

- MDString containing PCSection S's section name

- An MDTuple entry with all data associated with the PCSection S
- Repeat for all PCSections associated with the instruction or function

The main hurdle with modifying this data format in-place is making sure that any modifications to the entry formats is propagated everywhere else in the codebase. This is because LLVM PCSections carry constants in the form of tuples, meaning that any modifications to the data rely on index offsets. If a field is added to or removed from a given data structure, any code that attempts to parse or modify this structure will need to be revised.

Information Loss from Instruction Optimization and Re-Emission

The rest of LLVM makes no guarantees on whether PCSections are preserved across optimization passes, instruction deletions or instruction replacements [Thec]. This is especially relevant for instruction-tied PCSections, which are often deleted when instructions are split, merged or removed. One particular example of this is the loop's canonical loop variable. In LLVM, this variable is referred to using a PHI node. PHI nodes, however, are known not to correspond directly to any machine-code concept and have to be lowered to register moves (or simply register allocation), and are then deleted entirely.

In general, any and all information about a loop that isn't about the location of any particular instruction is attached to the function rather than to any instruction. Our reasoning for this is that individual instructions may be copied, merged or deleted but the metadata attached to the function is relatively safe. So far, the only feasible way for this information to be lost is for the function itself to be optimized out and deleted, which also means that all loops contained within are deleted, meaning that the information inside those PCSections is no longer necessary anyhow.

One particular consequence of this is that since eBPF conditional jumps have comparisons built into them, the LLVM IR latch comparison instruction and the conditional jump instruction are, in some cases, merged together in machine IR. Manual testing shows that when this happens, the PCSections of the comparison instruction are lost, while the resulting latch-compare-and-jump instruction inherits the conditional jump's PCSections.

Deletions, however, are not the only thing that may compromise the the fidelity of the information carried by PCSections. For instance, when a conditional branch is tagged in LLVM IR as a latch jump instruction for a loop, later conversion to Machine IR may result in multiple machine instructions being emitted, which all inherit the jump-latch's PCSections [Thec]. In order to ensure that the correct instruction is tagged, we apply the following method:

- Each instruction tag initially contains the LLVM IR opcode of the IR instruction being tagged in the higher level loop tagging pass
- The low-level, pre-emit pass BPFPCSectionFixupPass later iterates over every machine instruction in the program, checking every instruction with an instruction-tagging PCSection.
- For each such instruction, the LLVM opcode is checked against the machine instruction's functionality. If, for example, the LLVM opcode corresponds to a conditional jump and the machine instruction isn't a conditional jump, the instruction tag is removed from the instruction.
- The opcode information is stripped from the instruction tags, making this process transparent to programs that parse the resulting PCSections.

This, and the fact that conditional branches in general are very unlikely to be optimized out are the reasons that the compare-jump instruction is chosen to be the "anchor" by which a loop certificate metadata structure is tied to a specific point in the program, within the loop. In particular, the naive approach of pointing to the loop's beginning fails because it requires the instruction at the very start of the loop to stay stable and not be optimized out, which makes the tagging mechanism unreliable.

5.2.2 PCSections as Output Medium

Since each metadata entry is emitted together with a pointer to the relevant function or instruction, metadata is directly tied to the instruction or function it is related to. However, the instruction offsets are only emitted after all LLVM passes are already done. Therefore, the passes that generate the metadata do not have direct access to the instruction offsets themselves. Therefore, if a larger table like the main Loop Classification table needs to refer to a specific instruction, it must do so by assigning the instruction a PCSection with a module-unique ID number, and using the ID number as the reference to the instruction.

For a reference on how the resulting output may be decoded, see Section 5.5.

5.2.3 Additional Scaffolding

Each instruction or function may only have one PCSection entry per section. As a result, in order to be able to attach multiple metadata entries of identical structure to one PCSection entry, we have utilized an approach similar to a Pascal string: the number of entries encoded as a 64 bit unsigned integer, followed by the entries. However, in order for this structure to be parse-able, each individual entry has to either have a preset size,

or has to provide a way for its size to be deciphered during the parsing stage. For ease of use, our project additionally implements scaffolding infrastructure that handles the creation and management of these PCSection arrays.

5.3 Testing Infrastructure

5.3.1 Automated Parsing

In order to ensure that the emitted metadata is accurate and parseable, we additionally implement parsers for all emitted PCSections in ImHex [2], which allows one to visualize the parsing process and see what metadata is being emitted for which program, allowing direct detection of feature regressions and providing a template for those wishing to implement a parser for our data structures.

5.3.2 Unit Tests

In addition to binary inspection tools, we also implement a PCSection parser in Python, which not only parses all emitted PCSections in any given binary, but also cross references them to see if the overall structure is valid. For example, loop measures are only expected to arrive if the loop's main classification table states that they are, instruction tags are expected to refer to instructions inside the same function as the loop, and instruction IDs are expected to be valid. These, and many other integrity properties of the overall metadata structure, are automatically checked for a number of simple test cases. More details on the test cases and results are given in section 6.1.2

5.4 Practical Limitations

Due to how heavily our project relies on LLVM's own loop analysis infrastructure, the limitations of this loop analysis infrastructure translates to severe limitations on the types of loops we can verify:

- Since LLVM's loop boundary calculation and therefore loop variable retrieval utilities only work on loops with well defined end and constant stride values, our verification capabilities are limited to these cases for now.
- Since Scalar Evolution / the loop boundary calculator does not recognize GetElementPointer instructions as increment instructions, loops where the loop variable is itself a pointer cannot be verified using our current infrastructure.

[2] Also available in the repository mentioned in this section's beginning

- Using PCSections to emit information means that we can only "tag" instructions that we know are going to make it to the final binary, lest we lose the PCSection completely. This means that we cannot simply tag the "beginning" of any given loop with a PCSection in order to definitively state its location. The verifier must use the latch entries to infer the loop's location.
- PCSections only emit PCs after all passes are complete, which means that it is impossible to put PCs directly inside PCSections. This restriction means that it is largely impossible to have all of the metadata be contained in one single section or table, necessitating multiple sections of metadata that may reference one another.
- Loop measures where the end-boundary contains a PHI expression do not include information about which execution path leads to which value, since PHI nodes' entering blocks are LLVM basic blocks which are completely lost during emission to binary, and are therefore untaggable.
- As seen in Subsection 6.1.6, more complex compilation processes currently result in unparseable certificate sections. This is most likely due to a bug in the way we handle emitting PCSections.

5.5 Output Encoding

Once all passes in our implementation have finished, the resulting PCSections are emitted into the resulting ELF binary according to the following format:

5.5.1 Loop Tagging

Field Type and Size	Field	Description
u32	function_pointer	
u32	function_length	
u64	entry_count	
Entry[entry_count]	entries	Loop database entries, see table 5.2

Table 5.1: Loop Tagging PCSection format

Field Type and Size	Field	Description
u64	loop_id	Unique-per-module loop ID
u64	start	Start value of the IV if available
u64	finish	End value of the IV if available
u64	stride	Stride value of the IV if available
u64	exact_exit_count	Number of times the loop is calculated to run
u64	const_max_exit_count	Maximum number of times the loop may run
u64	branch_id	ID of the loop's cond. jump, see table 5.20
u64	compare_id	ID of the loop header's comparison, see 5.20
u16	comparison_type	Cond. branch comparison type, see tab. 5.3
u8 (bool)	isAscending	Direction of the loop variable
u8 (bool)	loopTerminates	1 if guaranteed to terminate, 0 if not.
u8 (bool)	hasIsAscending	is isAscending valid?
u8 (bool)	hasStart	is the *start* field valid?
u8 (bool)	hasFinish	is the $finish$ field valid?
u8 (bool)	hasStride	is the *stride* field valid?
u8 (bool)	hasLoopTerminates	is loopTerminates valid?
u8 (bool)	hasExactExitCount	is exact_exit_count valid?
u8 (bool)	hasConstMaxExitCount	is const_max_exit_count valid?
u8 (bool)	hasTranslationTableMeasure	1 if this loop has a measure, see table 5.4
u8 (bool)	hasBranchID	Is branch_id valid?
u8 (bool)	hasCompareID	Is compare_id valid?
u64	latch_count	How many loop latches are there?
u64[latch_count]	latches	IDs of loop latch instructions, see table 5.20

Table 5.2: Loop Tagging PCSection Entry format

The main table used to describe loops. The relevant fields for a proof verifier are the *latches* and *hasTranslationTableMeasure* fields, as well as the *loop_id* field to retrieve related entries from other tables. All other information contained in this table is for debugging purposes, to see what information LLVM itself is gathering about the loop behind the scenes.

Enum Values	Meaning
FALSE = 0	$false$
GT = 1	$>$
EQ = 2	$=$
GEQ = 3	$\geq$
LT = 4	$<$
NEQ = 5	$\neq$
LEQ = 6	$\leq$
TRUE = 7	$true$
C_UNK = 8	Unknown loop comparison

Table 5.3: Loop Comparison Types

Loop Measure Tagging

Field Type and Size	Field	Description
u32	function_pointer	
u32	function_length	
u64	entry_count	
LoopEntry[entry_count]	entries	See table 5.5

Table 5.4: Symbounds PCSection format

Field Type and Size	Field	Description
u64	loop_id	Loop ID, see table 5.1
u64	str_len	Length of format string
u8[str_len]	format_string	Format string. See section 5.1.3
u64	dependency_count	Number of dependencies
Dependency[dependency_count]	deps	Dependencies, see tables 5.6, 5.7, 5.8

Table 5.5: Symbounds PCSection LoopEntry format

The main table used to describe loop measures. The loop measure string is encoded in *format_string* as described in section 5.1.3, with the dependencies being listed in order in the *deps* field.

Field Type and Size	Field	Description
u64	type = 0	Type value for Instruction Dependency
u64	inst_tag_id	Instruction ID, see table 5.20

Table 5.6: Instruction Dependency

Encodes a dependency to the output value of an instruction.

Field Type and Size	Field	Description
u64	type = 1	Type value for Constant Dependency
u64	constval_size	Size of embedded constant, in bytes
If constval_size = 1:		
u8	data	Constant data
If constval_size = 2:		
u16	data	Constant data
If constval_size = 4:		
u32	data	Constant data
If constval_size = 8:		
u64	data	Constant data
If none of the above apply:		
u8[constval_size]	data	Constant data as byte array

Table 5.7: Constant Dependency

Encodes an integer constant.

Field Type and Size	Field	Description
u64	type = 2	Type value for Argument Dependency
u64	arg_no	Function argument number

Table 5.8: Argument Dependency

Encodes a dependency to one of the function's arguments.

5.5.2 Function Type Tagging

Enum Value	Description
HalfTyID (= 0)	16-bit floating point type
BFloatTyID (= 1)	16-bit floating point type (7-bit significand)
FloatTyID (= 2)	32-bit floating point type
DoubleTyID (= 3)	64-bit floating point type
X86_FP80TyID (= 4)	80-bit floating point type (X87)
FP128TyID (= 5)	128-bit floating point type (112-bit significand)
PPC_FP128TyID (= 6)	128-bit floating point type (two 64-bits, PowerPC)
VoidTyID (= 7)	type with no size
LabelTyID (= 8)	Labels
MetadataTyID (= 9)	Metadata
X86_AMXTyID (= 10)	AMX vectors (8192 bits, X86 specific)
TokenTyID (= 11)	Tokens
IntegerTyID (= 12)	Arbitrary bit width integers
ByteTyID (= 13)	Arbitrary bit width bytes
FunctionTyID (= 14)	Functions
PointerTyID (= 15)	Pointers
StructTyID (= 16)	Structures
ArrayTyID (= 17)	Arrays
FixedVectorTyID (= 18)	Fixed width SIMD vector type
ScalableVectorTyID (= 19)	Scalable SIMD vector type
TypedPointerTyID (= 20)	Typed pointer used by some GPU targets
TargetExtTyID (= 21)	Target extension type

Table 5.9: Type tagging TypeID Enum

Enum of types which may be encountered in a type encoding. Lifted directly from LLVM's own LLVM IR Type enum.

Field Type and Size	Field	Description
u32	function_pointer	Offset of function
u32	function_length	Length of function
TypeField	type	Type signature of function. See table 5.13

Table 5.10: Function Type Annotation PCSection format

Main metadata table for function type annotation. Each function in the program gets its own type signature. The type signature may be arbitrarily complex. Aggregate or complex types such as structs, functions and arrays contain type entries for their constituents. Type entries are polymorphic, with the parser being expected to read the first 64 bits where a type entry is expected to be found, and then use the type ID stored within to determine which template to use to parse the type contained within.

Type Entries

Field Type and Size	Field	Description
u64(TypeID)	type_id=IntegerTyID	Type ID.
u64	int_bit_width	Integer bit width

Table 5.11: Integer TypeField

Field Type and Size	Field	Description
u64(TypeID)	type_id=ByteTyID	Type ID.
u64	byte_bit_width	Byte bit width

Table 5.12: Byte TypeField

Field Type and Size	Field	Description
u64(TypeID)	type_id=FunctionTyID	Type ID.
bool(u64)	is_vararg	Does this function accept varargs?
TypeForm	return_type	Return type
u64	param_count	How many args does this function have?
TypeForm[param_count]	param_types	Type of each argument

Table 5.13: Function TypeField

Field Type and Size	Field	Description
u64(TypeID)	type_id=StructTyID	Type ID.
bool(u64)	is_packed	Is this struct packed?
bool(u64)	is_opaque	Is this struct opaque?
The following entries are only present if is_opaque=false		
u64	entry_count	How many entries does this struct have?
TypeForm[param_count]	param_types	Type of each struct entry

Table 5.14: Struct TypeField

Field Type and Size	Field	Description
u64(TypeID)	type_id=ArrayTyID	Type ID.
u64	element_count	Array element count
TypeField	array_elem_type	Type of elements contained by array

Table 5.15: Array TypeField

Field Type and Size	Field	Description
u64(TypeID)	type_id=FixedVectorTyID	Type ID.
u64	element_count	Vector element count
TypeField	vector_elem_type	Type of elements contained by vector

Table 5.16: Fixed Vector TypeField

Field Type and Size	Field	Description
u64(TypeID)	type_id=ScalableVectorTyID	Type ID.
TypeField	vector_elem_type	Type of elements contained by vector

Table 5.17: Scalable Vector TypeField

Field Type and Size	Field	Description
u64(TypeID)	type_id=TypedPointerTyID	Type ID.
TypeField	pointer_elem_type	Type pointed to

Table 5.18: Typed Pointer TypeField

Field Type and Size	Field	Description
u64(TypeID)	type_id	Type ID. Not one of the special cases.

Table 5.19: Generic TypeField

Encodes all types not covered in the tables above.

5.5.3 Instruction Tagging

Field Type and Size	Field	Description
u32	inst_pointer	Pointer to the instruction, as an offset inside the ELF file
u64	instr_tag_id	Unique-in-module ID number assigned to instruction

Table 5.20: Instruction Tagging PCSection format

Main entry type for matching instructions to instruction IDs. Every instruction that all other tables refer to are to be found in this section.

5.5.4 Register Tagging

Enum Values	NoRegister = 0
R0 = 1	W0 = 13
R1 = 2	W1 = 14
R2 = 3	W2 = 15
R3 = 4	W3 = 16
R4 = 5	W4 = 17
R5 = 6	W5 = 18
R6 = 7	W6 = 19
R7 = 8	W7 = 20
R8 = 9	W8 = 21
R9 = 10	W9 = 22
R10 = 11	W10 = 23
R11 = 12	W11 = 24

Table 5.21: BPFRegister Encoding Enum

BPF registers mapped to numbers. Here, R0-R11 refer to registers 0-11 accessed in 64 bit mode, while W0-W11 refer to those same registers being accessed by 32 bit instructions.

Field Type and Size	Field	Description
u32	function_pointer	Pointer to the function
u32	function_length	Function length
u64	entry_count	Entry count
RegisterTaggingEntry[entry_count]	entries	Register tagging entries, see table 5.23

Table 5.22: Loop Variable Register Tagging PCSection format

Field Type and Size	Field	Description
u64	loop_id	Loop ID. See table 5.1
u32(BPFRegister)	iv_register	Register that the loop variable is stored in. See table 5.21

Table 5.23: RegisterTaggingEntry format

Main entry format for register tagging. For each loop tagged by the proof generator, its corresponding entry in this table states which register the loop variable lives in when entering the loop body.

6 Evaluation

In this chapter, we evaluate the suitability of our augmentation suite for real world use. Due to the absence of an SMT solver based proof verifier at this time, evaluation of certificate validity is done manually where possible. [1]

6.1 Testing

6.1.1 Criteria

In order to judge suitability for real world use, we evaluate test cases by the following criteria:

- Certificate presence: Could our suite generate a sufficiently full certificate, defined as one possessing a loop measure and all dependencies required to parse it?
- Certificate validity: Is the certificate valid?
- Metadata footprint: How large is the resulting metadata? What percentage of the resulting binary is our certificate, by data size?

Since our compile-time passes are on the order of O(N) with respect to the instruction count, we do not evaluate compilation time during our testing.

6.1.2 Simple Unit Tests

We begin our evaluation with the simplest test cases. Here, we test the basic functionality of our implementation.

Test - AscendingGeneric

```
int hello_world(void *ctx) {
  for (unsigned int i=0;i<5000;i++) {
    bpf_printk("ascending␣loop,␣terminates␣%d\n", i);
```

[1] Our test suite and parsing toolset is available at `https://github.com/TUM-DSE/tnbpf-tooling`

```
  }
  return 0;
}
```

The simplest possible loop type. The resulting measure, after filling in the dependency entries and simplifying, resolves to $5000 - i$, which is trivially a valid measure.

Test - DescendingGeneric

```
int hello_world(void *ctx) {
  for (unsigned int i=5000;i>0;i--) {
    bpf_printk("descending␣loop,␣terminates␣%d\n"i i);
  }
  return 0;
}
```

The resulting measure is i, a trivially valid measure.

Test - AscendingStride

```
int hello_world(void *ctx) {
    for (unsigned int i=0;i<5000;i+=3) {
      bpf_printk("ascending␣loop␣<␣stride␣3␣%d\n", i);
    }
    for (unsigned int i=0;i<5000;i+=5) {
      bpf_printk("ascending␣loop␣<␣stride␣5␣%d\n", i);
    }
    return 0;
}
```

Here, we have loops with various stride values. The first loop has the measure $4997 - i$ and the second loop has $4995 - i$. Both trivially valid.

6.1.3 Complex Unit Tests

Test - PhiEnd

```
int hello_world(void *ctx) {
  unsigned int *unk = (unsigned int *)ctx;
  unsigned int a;
  a = unk[0];
```

```
  int n = 10;
  if (a) {
    bpf_printk("branch␣side␣effect\n");
    n = 20;
  }

  for (int i=a;i<n;i++) {
    bpf_printk("phi␣loop␣!=\n");
  }
  return 0;
}
```

For this loop, the boundary value is not one single constant integer, and instead may take on the values 10 or 20, depending on the program's execution flow. Here, the generated measure is $(phi\ 20\ 10) - i$. Although this particular measure may be possible to reduce down to $20 - i$ via edge case handling, a more general method of handling phi nodes in the verifier may not be possible. For this case, we recommend further research, see section 8.2.

Test - WrongWaySigned

```
int hello_world(void *ctx) {
  for (int i=50;i>10;i++) {
    bpf_printk("Will␣overflow␣and␣UB\n");
  }
  return 0;
}
```

Since LLVM assumes that signed overflows and underflows are undefined behaviour in C, and works on the assumption that they will never occur [Thea, Chapter 6.3.2], the result is that our augmentation passes fail to generate any measure whatsoever. In this case, this is the correct behaviour.

Test - WrongWayUnsigned

```
int hello_world(void *ctx) {
  for (unsigned int i=50;i>10;i++) {
    bpf_printk("%d\n", i);
  }
  return 0;
```

```
}
```

For this case, the proof generator wrongfully generates the measure $-i$, but correctly deduces the end-value of 0. Although the loop can be verified with the measure $UINTMAX - i$, the certificate generated by the proof generator is invalid due to the measure being negative during at least one loop execution.

Test - PointerLoop

```
int hello_world(void *ctx) {
  int *array = (int *)(ctx);
  int *array_end = &array[50];
  for (;array < array_end; array = &array[1]) {
    bpf_printk("array␣loop␣%d\n", *array);
  }
}
```

In this case, the proof generator fails to generate any measure, despite the fact that LLVM's IR optimizations optimize the loop into a for-loop with the boundary-value 200 and stride 4. In this case, a measure of $array_end - array$ could theoretically be verifiable, if the proof verifier can deduce that the element-pointer calculation for *array_end* will not overflow.

Test - VariableStride

```
int hello_world(void *ctx) {
  int *unk = (int *)(ctx);
  for (int i=0;i<20;) {
    bpf_printk("side␣effect␣%d\n", i);
    if (unk[i] & 1) {
      i++;
    } else {
      i += 2;
    }
  }
  return 0;
}
```

For this test case, the proof generator also fails to generate any certificate at all, due to the loop stride not being loop invariant. However, the lack of overflows and the

loop condition of $i < 20$ means that $20 - i$ is a valid measure. This particular case failing to verify is a limitation of LLVM's loop analysis tools, which fail to provide loop boundaries in any form if the stride is non-loop-invariant.

Test - NestedRegular

```
int hello_world(void *ctx) {
  for (int i=0;i<100;i++) {
      for (int j=0;j<100;j++) {
          bpf_printk("%d␣%d\n", i, j);
      }
  }
  return 0;
}
```

For this test case, the proof generator emits the measure $100 - i$ and $100 - j$ for the outer and inner loops respectively. However, while j is correctly tagged with its register, i is not. Therefore, this certificate would most likely fail to verify, in spite of the measures themselves being valid.

Test - NestedInterdependent

```
int hello_world(void *ctx) {
  for (int i=10;i<100;i++) {
      for (int j=0;j<i;j++) {
          bpf_printk("%d␣%d\n", i, j);
      }
  }
  return 0;
}
```

In this test case, the outer loop is assigned the measure $100 - i$, but i is not associated with a register. The inner loop, however, gets a register mapping for j while the emitted loop measure is faulty. This proof would also fail to verify, despite the fact that $i - j$ would have been a valid measure for the inner loop.

6.1.4 Edge-Case Testing Unit Tests

Test - EdgecaseDescendingMiss

```
int hello_world(void *ctx) {
  for (unsigned int i=5;i>0;i-=2) {
    bpf_printk("5␣not␣even,␣this␣will␣never␣be␣exactly␣zero␣%d\n", i);
  }
  return 0;
}
```

Here, the proof generator falsely generates the measure i, which would fail to verify because i would underflow and wrap back around, meaning that the value of i increases. However, checking cases like these requires modular arithmetic best left to full-fledged proof verifiers.

Test - EdgecaseAscendingMiss

```
int hello_world(void *ctx) {
  for (unsigned int i=0;i<=0xfffffffe;i+=2) {
    bpf_printk("ascending␣loop,␣does␣not␣terminate␣%d\n", i);
  }
  return 0;
}
```

For this case, the proof generator correctly fails to generate a measure, as the necessary condition for termination ($i = 0xffffffff$) is unreachable due to i staying even (the lowest bit stays equal to 0) through the whole operation, and $0xffffffff$ being odd. However, the actual reason proof generation fails is that LLVM optimizes this loop into an infinite loop in the IR optimization stage, before the backend is run, causing the proof generator to fail to obtain loop bounds.

Test - EdgecaseUnsigned

```
int hello_world(void *ctx) {
  for (unsigned int i=0;i!=5;i+=2) {
    bpf_printk("will␣not␣terminate,␣5␣is␣odd,␣i␣is␣always␣even␣%d\n", i);
  }
  return 0;
}
```

Here, the proof generator correctly fails to generate a measure, due to LLVM turning the loop into an infinite unconditional loop at the IR level.

Test - SignedUnsigned

```
int hello_world(void *ctx) {
  unsigned int bound = 0x80000001;
  for (int i = 0;i<bound;i++) {
    bpf_printk("Infinite␣loop,␣i␣will␣overflow␣%d\n", i);
  }
  return 0;
}
```

Here, the proof generator also correctly fails to generate a measure, as LLVM turns the loop into an infinite unconditional loop.

6.1.5 Metadata Footprint of Test Cases

In the following table, the total size of each program proof we generate is given, in relation to both the executable section of the program and the overall ELF binary.[2]

Test Name	Metadata Size	Size wrt. Code Segment	Percent of Binary
AscendingGeneric	303.0 B	344.32%	4.79%
AscendingStride	502.0 B	348.61%	7.03%
DescendingGeneric	249.0 B	282.95%	3.96%
EdgecaseAscendingMiss	178.0 B	445.00%	3.14%
EdgecaseDescendingMiss	249.0 B	282.95%	3.92%
EdgecaseUnsigned	178.0 B	278.12%	3.06%
NestedInterdependent	546.0 B	455.00%	8.16%
NestedRegular	502.0 B	392.19%	7.56%
PhiEnd	346.0 B	254.41%	4.93%
PointerLoop	178.0 B	171.15%	2.93%
SignedUnsigned	178.0 B	278.12%	3.06%
VariableStride	178.0 B	111.25%	2.89%
WrongWaySigned	178.0 B	445.00%	3.12%
WrongWayUnsigned	276.0 B	215.62%	4.20%

Table 6.1: Metadata Footprint of Overall Certificate Generation Process

[2]The graphs were generated using matplotlib. The Python code used to generate them is available at `https://github.com/TUM-DSE/tnbpf-tooling`

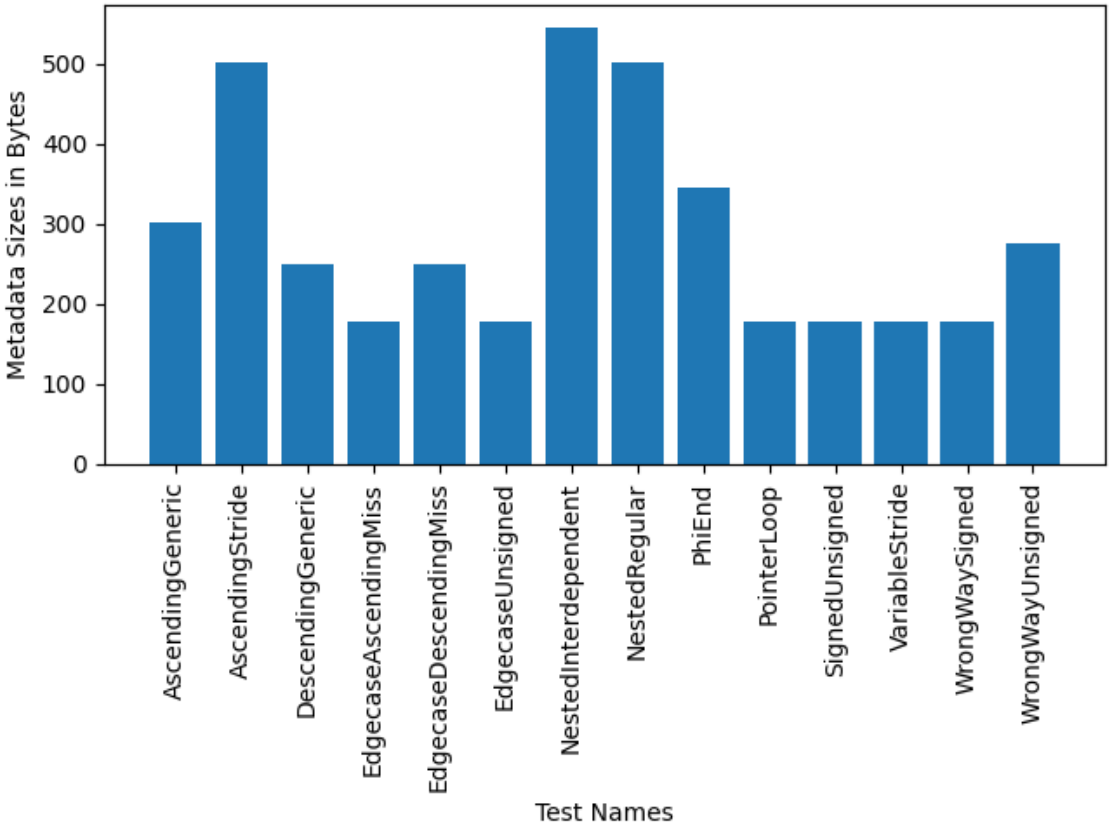


Figure 6.1: Metadata Sizes of Test Cases

Overall, the proofs we generate span hundreds of bytes. In tests involving one single loop per program, our proof generator appends approximately 100-300 bytes of information to the program. In tests involving nested loops, this total size goes up to 400-600 bytes. From this, we may approximate a binary size footprint in the order of 200-300 bytes per loop.

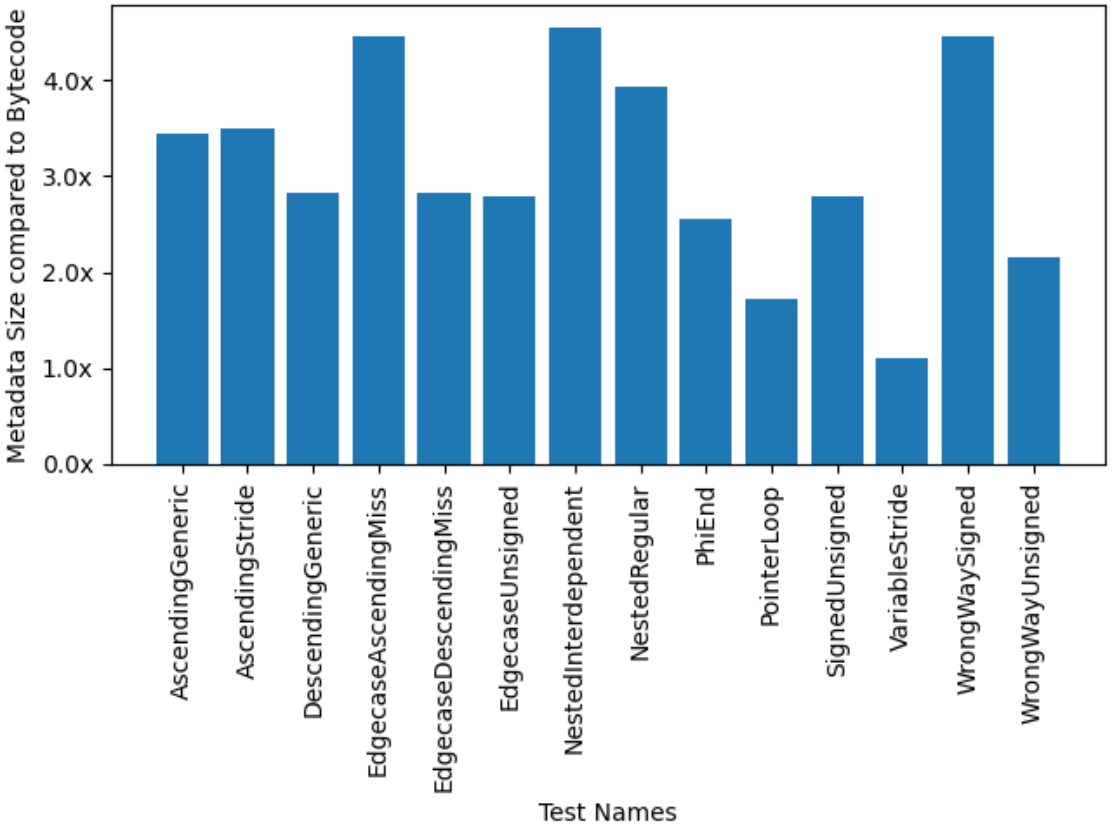


Figure 6.2: Metadata Footprints compared to Code Segments

For each test case, we additionally computed the ratio of proof metadata to actual eBPF bytecode. For our test cases, we find that our proof generator's certificates range from being the same size as the executable bytecode itself to being five times its size. The median certificate in our test suite was approximately 2.8 times the size of the executable bytecode bundled in the ELF binary. These numbers may seem large at first glance, but our test cases involve functions with only one loop in their body and nothing else. This loops-per-program density is much higher than what is typical for eBPF modules in production, as seen in the example projects listed in section 6.1.6.

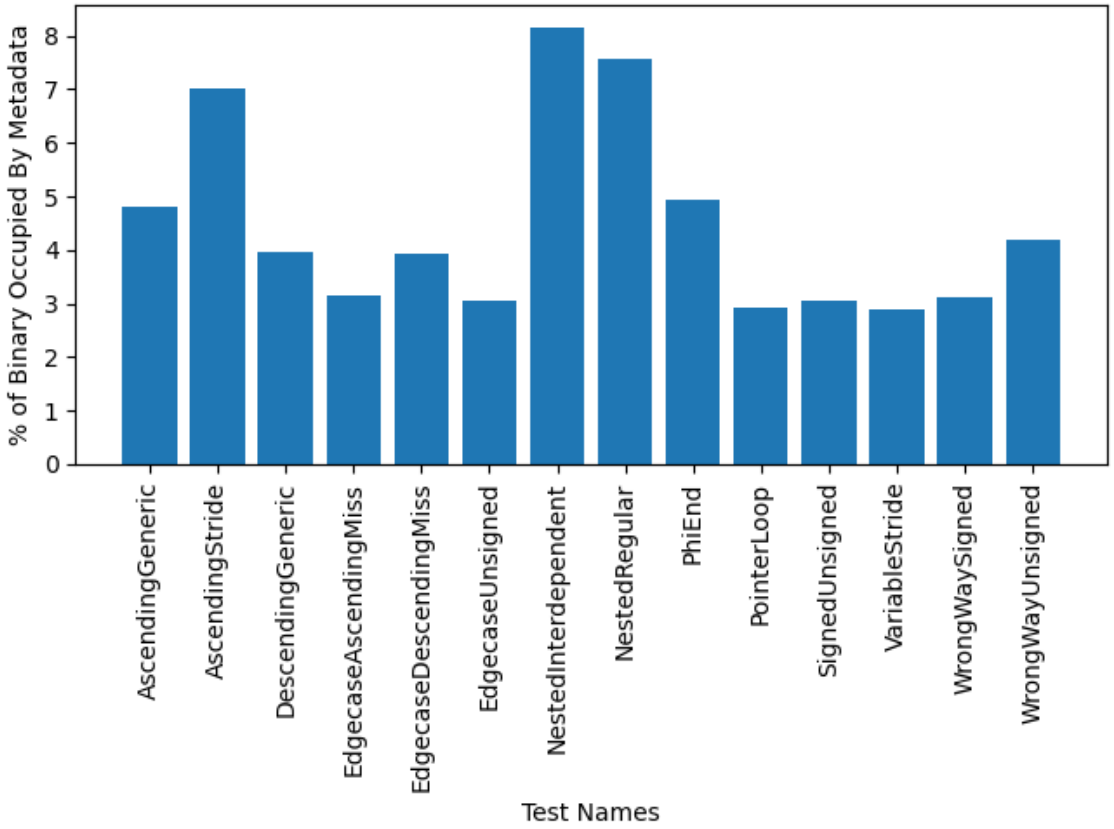


Figure 6.3: Percentages of Total Binary Occupied by Metadata

In addition to the two previous datapoints, we also measured what percent of the resulting ELF binary object our proof certificates occupied. Even for our particularly loop-heavy test cases, our metadata was dwarfed by the overall size of the ELF binary itself, with the largest certificates taking up approximately 8 percent of the ELF file. Although further testing with larger programs may be recommended, it is worth noting that the average eBPF binary is in the order of several megabytes. This means that even if this ratio were closer to 75 percent, meaning that the overall file size were quadrupled, this proof generator would still remain practical, as these proof certificates only need to stay in memory until the verifier is done checking them.

6.1.6 Testing on production eBPF modules

In addition to our own minimal test cases, we tested our proof generator on the Cilium team's Cilium ([3]) and Tetragon ([4]) projects. However, the Cilium project did not include any loops, while the Tetragon project produced duplicate PCSection entries. We suspect that this is due to the way Tetragon links its eBPF modules. However, further testing remains recommended, see section 8.2.

6.1.7 Results

Overall, our proof generator appears to be functional in simpler loops, while unsigned wraparounds present some difficulty for the measure generation algorithm, as a valid measure expression would require unsigned subtraction with overflow and underflows built in, since the loop variable itself wraps around. As the test cases PointerLoop and VariableStride show, the proof verifier remains narrow scoped and limited in its ability to recognize more complex loop types.

6.1.8 Notes on Loop Termination In Practice

For the purposes of our evaluation, we only check whether or not the loop will eventually terminate, given infinite time. However, this criterium would still allow a loop like

```
for (uint64_t i=0;i<UINT64_MAX;i++) {
    ...
}
```

to verify successfully. Although a loop like this would, theoretically, terminate, it should still be taken into consideration that the primary reason for checking loop-termination is to ensure that the eBPF module eventually relinquishes the CPU to the rest of the kernel, to ensure the responsiveness of the operating system. In the current verifier, the approach of simulating every loop execution means that the calculated cost is approximately proportional to the loop's execution duration, which in turn means that loops that take too long to execute are also disqualified. Our approach, however, could verify termination for arbitrarily long-lasting loops, as long as they are finite. In practice, however, a loop (or chain of nested loops) that takes several trillion years to terminate would have the same practical effects on the kernel as a loop that never terminates.

[3] https://github.com/cilium/cilium
[4] https://github.com/cilium/tetragon

7 Related Work

7.1 Loop Measures and Verification

For our loop verification measures, we use a similar approach to the Verus project, a general program verifier for Rust, whose *decreases* clause for loop and recursive function termination closely mirrors the functionality provided by our loop measures [Lat+23]. A similar approach to loop termination is also used in Dafny, a programming language and IDE plugin family built to verify programs in C#, Java, Javascript, Go and Python [Lei10].

7.2 eBPF Proof Generation

The PCC generating eBPF-specific framework Exoverifier [Thed], the VeRefine [Ham+25] certificate generator prototype and the Prevail [Ger+19] verifier are all examples of projects built to tackle the existing eBPF verifier's limitations.

Exoverifier is a set of two proof generators that generate proofs of eBPF program safety, one of which uses abstract interpretation and the other symbolic execution. It uses the Lean prover to generate and export formal proofs in a standard format.

Prevail is a userspace verifier that verifies eBPF binaries using static analysis, but does not generate a proof, instead optionally emitting invariants [Ham+25]. However, it still has better verification runtime guarantees than the current eBPF verifier.

VeRefine builds on Prevail by implementing a type inference based approach to annotate each instruction in an eBPF program's bytecode. Unlike its predecessor, it also separates proof generation from verification using a PCC based approach.

There also exists work on verifying termination non-nested linear loops by generating ranking functions [PR04], but this work remains limited due to its inability to handle nested loops.

8 Conclusion

8.1 Summary Of Results

By integrating LLVM's own loop analysis infrastructure, we attached a loop certificate generator to the LLVM eBPF backend. This certificate generator is able to provide termination proofs for various types of loops, which may then be checked by an external verifier.

However, testing indicates that some loop types like nested loops remain outside the ability of our current code to verify, despite the relevant infrastructure being in place.

8.1.1 Practical Considerations

During the testing phase of our certificate generator's development, we additionally noticed that eBPF modules used in production sometimes don't include any loops at all. Although this may be due to eBPF modules themselves being simple by design, we also attribute this partially to the fact that support for loops in the current eBPF verifier is relatively new, with Linux kernel version 5.3 introducing the capability in 2019 [eBPb]. Therefore, as verification capability expands, we expect loops to gain more prevalence in eBPF modules.

8.2 Future Work

At the time of writing, the proof generator we implemented remains a prototype with severe limitations.

First and foremost, it would be preferrable for the proof generator to emit a more standardized, proof adjacent structure confined to one single section rather than a slew of PCSections. Furthermore, we believe it to be necessary to clean up the codebase, making sure it conforms to LLVM's own norms and interface contracts. For example, the current codebase has references to BPF-specific PCSections in the PhiElimination pass that should be backend-agnostic.

Also, due to the way PHI nodes are encoded in our measure strings (See section 5.4), the only thing a verifier may do upon encountering one is attempt to generate a measure that works for all possible values the PHI node may take. A way to embed

information about which execution-flows lead to which values into the measure string would enable a more standard verification approach.

More importantly, the current proof generator only proves loop termination, but doesn't necessarily make it easier to generate safety proofs regarding what's going on inside the loop. Integrating SCEV for this purpose would probably help greatly, since it computes arbitrary values at arbitrary points of loop execution.

Moreover, our implementation necessitated home-grown PCSections tooling, for example to store multiple instances of a metadata structure connected to the same PCSection entry. Our current solution is to store them in a Pascal string style array. This structure emerging organically shows the necessity of more formalized tooling surrounding PCSections to facilitate projects like these.

Additionally, as discussed in section 6.1.8, merely proving termination may not be sufficient for practical use. The build-in cost limit of the current verifier also limits execution time, but our approach doesn't. Additional cost-calculating measures may be required.

8.2.1 Testing

In general, we recommend anyone basing their work on our codebase to handle more classes of loops inside the test suite as well as implement automated regression testing.

Furthermore, our system is designed to be language agnostic and work on anything that compiles via LLVM IR. But in our evaluation, we only test C source code. Further testing with other eBPF-compatible languages like Rust is recommended.

In addition, production-grade eBPF modules from the Tetragon project currently cause our proof generator to emit duplicate PCSection entries. Further testing to ascertain why, and then alleviation of the root cause remain necessary.

Abbreviations

ISA Instruction Set Architecture

IR Intermediate Representation

PCC Proof-Carrying Code

JIT Just In Time

FUSE Filesystem in Userspace

SCEV Scalar Evolution

List of Figures

3.1 Overview of our envisioned workflow 6
3.2 Overview of our implementation 7

6.1 Metadata Sizes of Test Cases 41
6.2 Metadata Footprints compared to Code Segments 42
6.3 Percentages of Total Binary Occupied by Metadata 43

List of Tables

5.1 Loop Tagging PCSection format 25
5.2 Loop Tagging PCSection Entry format 26
5.3 Loop Comparison Types 27
5.4 Symbounds PCSection format 27
5.5 Symbounds PCSection LoopEntry format 27
5.6 Instruction Dependency 28
5.7 Constant Dependency 28
5.8 Argument Dependency 28
5.9 Type tagging TypeID Enum 29
5.10 Function Type Annotation PCSection format 29
5.11 Integer TypeField 30
5.12 Byte TypeField 30
5.13 Function TypeField 30
5.14 Struct TypeField 31
5.15 Array TypeField 31
5.16 Fixed Vector TypeField 31
5.17 Scalable Vector TypeField 31
5.18 Typed Pointer TypeField 31
5.19 Generic TypeField 32
5.20 Instruction Tagging PCSection format 32
5.21 BPFRegister Encoding Enum 32
5.22 Loop Variable Register Tagging PCSection format 33
5.23 RegisterTaggingEntry format 33

6.1 Metadata Footprint of Overall Certificate Generation Process 40

Bibliography

[eBPa] eBPF Community. *Functions - eBPF Docs*. https://docs.ebpf.io/linux/concepts/functions/. Accessed on 18.09.2026.

[eBPb] eBPF Community. *Loops - eBPF Docs*. https://docs.ebpf.io/linux/concepts/loops/. Accessed on 16.09.2026.

[Flo93] R. W. Floyd. "Assigning Meanings to Programs." In: *Program Verification: Fundamental Issues in Computer Science*. Ed. by T. R. Colburn, J. H. Fetzer, and T. L. Rankin. Dordrecht: Springer Netherlands, 1993, pp. 65–81. ISBN: 978-94-011-1793-7. DOI: 10.1007/978-94-011-1793-7_4.

[Ger+19] E. Gershuni, N. Amit, A. Gurfinkel, N. Narodytska, J. A. Navas, N. Rinetzky, L. Ryzhyk, and M. Sagiv. "Simple and precise static analysis of untrusted Linux kernel extensions." In: *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*. PLDI 2019. Phoenix, AZ, USA: Association for Computing Machinery, 2019, pp. 1069–1084. ISBN: 9781450367127. DOI: 10.1145/3314221.3314590.

[Ham+25] A. Hamza, L. Zavalia, A. Gurfinkel, J. A. Navas, and G. Fedyukovich. "A Flow-Sensitive Refinement Type System for Verifying eBPF Programs." In: *Proc. ACM Program. Lang.* 9.OOPSLA2 (Oct. 2025). DOI: 10.1145/3763799.

[LA04] C. Lattner and V. Adve. "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation." In: *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization*. CGO '04. Palo Alto, California: IEEE Computer Society, 2004, p. 75. ISBN: 0769521029.

[Lat+23] A. Lattuada, T. Hance, C. Cho, M. Brun, I. Subasinghe, Y. Zhou, J. Howell, B. Parno, and C. Hawblitzel. "Verus: Verifying Rust Programs using Linear Ghost Types." In: *Proc. ACM Program. Lang.* 7.OOPSLA1 (Apr. 2023). DOI: 10.1145/3586037.

[Lei10] K. R. M. Leino. "Dafny: An Automatic Program Verifier for Functional Correctness." In: *Logic for Programming, Artificial Intelligence, and Reasoning*. Ed. by E. M. Clarke and A. Voronkov. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 348–370. ISBN: 978-3-642-17511-4.

[Nec97] G. C. Necula. "Proof-carrying code." In: *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL '97. Paris, France: Association for Computing Machinery, 1997, pp. 106–119. ISBN: 0897918533. DOI: 10.1145/263699.263712.

[PR04] A. Podelski and A. Rybalchenko. "A Complete Method for the Synthesis of Linear Ranking Functions." In: *Verification, Model Checking, and Abstract Interpretation*. Ed. by B. Steffen and G. Levi. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 239–251. ISBN: 978-3-540-24622-0.

[Sch+18] D. Scholz, D. Raumer, P. Emmerich, A. Kurtz, K. Lesiak, and G. Carle. "Performance Implications of Packet Filtering with Linux eBPF." In: *ITC (1)*. 2018, pp. 209–217.

[Sha+25] M. Shachnai, H. Vishwanathan, S. Narayana, and S. Nagarakatte. "Fixing Latent Unsound Abstract Operators in the eBPF Verifier of the Linux Kernel." In: *Static Analysis*. Ed. by R. Giacobazzi and A. Gorla. Cham: Springer Nature Switzerland, 2025, pp. 386–406. ISBN: 978-3-031-74776-2.

[Thea] The GNU Project. *GNU C Language Manual*. https://www.gnu.org/software/c-intro-and-ref/manual/html_node/index.html. Accessed on 19.09.2026.

[Theb] The Linux kernel development community. *BPF Documentation*. https://docs.kernel.org/bpf/index.html. Accessed 16.09.2026.

[Thec] The LLVM Project. *LLVM PC Sections Metadata*. https://llvm.org/docs/PCSectionsMetadata.html. Accessed on 18.09.2026.

[Thed] The UNSAT Group. *Exoverifier*. https://github.com/uw-unsat/exoverifier. Accessed on 16.09.2026.